DFRefactor

Table of contents

Welcome to DFRefactor

DFRefactor is a free, open-source tool for automated refactoring of DataFlex source code. It applies behavior-preserving transformations that bring legacy code in line with modern DataFlex syntax, improving readability and maintainability without changing program behavior.

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."
— Martin Fowler, Refactoring: Improving the Design of Existing Code (2nd edition, 2019)

DFRefactor is inspired by the discipline of refactoring as formalized in Martin Fowler's seminal work. See What Is Refactoring for background.

An online version of this help is available at Online help system and may be more current than the local CHM.

Components

DFRefactor ships as three programs:

Program	Audience	Purpose
`DFRefactor.exe`	All users	The main refactoring application. Selects functions, targets files or workspaces, runs transformations, reports results.
`TestBench.exe`	Advanced developers	A harness for developing new refactoring functions against a single test file. Refactored output can be compiled in-place to verify correctness.
`DFUnit_TestRunner.exe`	Advanced developers	The unit-test runner. Exercises `cRefactorFuncLib` (and its parent `cBaseFuncLib`) against fixtures in `oUnit_Tests.pkg`. Use it after any change to the function library to verify nothing has regressed. The DFUnit framework was originally developed by Ola Eldöy, enhanced by Bram NijenKamp, and adapted for this project.

Getting started

The intended workflow is:

1. Open a workspace (`.sws` file). DFRefactor needs a workspace for its backup logic, even when refactoring a single file.
2. Choose **Workspace** or **File** mode in the toolbar.
3. Select the refactoring functions to run from the function list.
4. Configure file extensions and source folders (Workspace mode only).
5. Click **Start Refactoring**.

For a detailed walk-through, see How to use the Program.

Project and contact

Source code, releases, and issue tracking: https://github.com/NilsSve/DfRefactor

- Bug reports: Issues
- Questions and suggestions: Discussions
- Commercial services (custom refactoring functions, full legacy-code migrations): see Feedback

What's new

Release notes for current and previous versions:

- DFRefactor 2026
- DFRefactor 2024 — the 2024 overhauled version (major architectural rewrite).
- DFRefactor 3.2.1.0 — final 3.x release before the 2024 overhaul.
- DFRefactor 3.1.0.0
- DFRefactor 3.0.0.1

DFRefactor 2026

This release rolls up the changes made since the 2024.0 architectural rewrite.

New refactoring functions:

- Legacy-grid to CodeJock conversions. A family of cross-file functions that modernise legacy grid and list objects:
  - `RefactorDbGridTocDbCJGrid` — data-aware `DbGrid` → `cDbCJGrid` with individual column objects.
  - `RefactorGridToCJGrid` — non-data-aware `Grid` → `cCJGrid` (property translation; columns completed manually).
  - `RefactorDbListTocDbCJGridPromptList` — `dbList` selection lists → `cDbCJGridPromptList` prompt lists.

  The data-aware and non-data-aware paths are implemented as separate classes for clarity and testability.

- Additional line-by-line functions:
  - `ChangeIfNotCommandToExpression`,
  - `ChangeLeftCommandToFunction`,
  - `ChangeRightCommandToFunction`,
  - `ChangeSquareBracketsIndicators`, and
  - `NormalizeArrayNotation`.
  - `ChangeIndexNumber`.
- New report function:
  - `ReportUnusedUseStatements` — flags `Use` statements whose imported classes are never instantiated (companion to `ReportUnusedSourceFiles`).

Architecture:

- The function-library class chain was restructured into focused, single-responsibility layers: `cRefactorUtilities` → `cRefactorLexer` → `cRefactorExpressionParser` → `cBaseFuncLib` → `cRefactorFuncLib` → `oRefactorFuncLib`.
- Global handles:
  shortened: `ghoRefactorEngine` → `ghoEngine`, `ghoRefactorFuncLib` → `ghoFuncLib`.
- Calling conventions unified between locally and globally defined functions.

DataFlex 2026 compatibility:

- The `cFilesystem` git submodule was removed. File I/O moved to a new `cRefactorFileIO.pkg` of DF26-compatible global functions — some of `cFilesystem`'s ANSI Win32 file calls crashed the DF26 debugger. `cRefactorFileIO` also absorbed DUF's channel-based file I/O, so the engine no longer routes routine file I/O through DUF (DUF

remains only for database migrations).
- Encoding-aware file read/write was centralized in `cEncodingHelpers.pkg`; its global functions are prefixed `RF` to avoid name clashes.
- Improved handling of DF26 `.sws` workspace-file reading.
- The earlier Studio Code Explorer / Code Completion issue with triple-quote `{ Description }` meta-tags is resolved in DF26.

Command-line and continuous integration:
- `DFRefactor.exe` now supports unattended/batch operation — workspace, single-file, folder, and batch flags, plus a /c ini-file mode. The current configuration can be written from Program Settings → Export to ini-file for repeatable runs. See Command Line Usage.
- `DFUnit_TestRunner.exe` gained a headless console mode (`-c` / `--console`) that runs the full suite and exits with code 0 (all pass) or `-1` (any fail), suitable for gating a CI build.

Testing and packaging:
- Substantially expanded unit-test coverage, including the grid/list conversions and the encoding helpers. Each test fixture now lives in its own file, `Used` into `oUnit_Tests.pkg`, so the suite stays maintainable.
- JSON Export/Import was reworked to match the new test structure. Functions in `UserDefinedRefactorFunctions.pkg` can be marked `{ Private = True }` to exclude them from JSON Export/Import.

DFRefactor 2024.0

The 2024.0 release is a major rewrite of DFRefactor. The previous concept proved too rigid to extend with new refactoring functions and was replaced with an engine-based architecture.

Architecture

- **New refactoring engine.** All source-code processing is handled by `cRefactorEngine`. The engine reads the **Functions** data table to identify selected functions, then dispatches to the matching implementation in the `cRefactorFuncLib` repository class (a subclass of `cBaseFuncLib`). The Functions table is registered through the **TestBench** program. The table's contents drive the **Function List** shown in the main UI.
- **Three-program split.** The single original program was split into `DFRefactor.exe` (end users), `TestBench.exe` (advanced users, function development), and `DFUnit_TestRunner.exe` (advanced users, automated unit tests).
- **`cBaseFuncLib`.** The new parent class for `cRefactorFuncLib` provides shared helpers. The most significant is the tokenizer, which converts a single source line into a struct array of tokens that refactoring functions can analyze robustly.
- **Project moved to GitHub.** Source, releases, and issue tracking now live at https://github.com/NilsSve/DfRefactor.

Supported DataFlex versions

DFRefactor 2024.0 supports the two most recent DataFlex Studio versions at the time of release: **DataFlex 2023 and 2024**. The tool can still be used to refactor projects originally created in earlier DataFlex versions, but one of the supported Studio versions must be installed to compile and run DFRefactor itself.

Workflow improvements

- **Per-workspace persistence.** Function selections and folder selections are saved per workspace and restored on the next session.
- **Automatic folder discovery.** When a new workspace is opened, all source sub-folders under the workspace home are automatically added to the **Select Folders** grid. The grid's right-click context menu adds, removes, and toggles folders. Combined with the **File Filter** drop-down, this defines the file set for each run.
- **Opening screen.** A Visual-Studio-style opening screen lets users pick a recent workspace or use the **Get Started** option. The current workspace can also be changed from the main toolbar at any time.
- **Bling controls.** Uncommenting `Define CI_ShowBlingStuff` near the top of `TestBench.src` or `DFRefactor.src` reveals the toolbar's Theme selector and Snapshot button.

Backup mechanism

- **Overwrite-cycle backups.** Although it remains the user's responsibility to check in source code before refactoring, DFRefactor uses a backup mechanism with a configurable overwrite cycle (99 days by default; settable in **Program Settings**). Within the cycle window the original source is preserved across multiple refactoring runs — the existing backup is not overwritten when applying a series of incremental changes. After the window expires, the next run creates fresh backups. This also enables in-cycle undo by moving backups back to their original locations.

TestBench

The TestBench program is intended for advanced developers writing new refactoring functions. It works with a single test file containing legacy code snippets. A small test program is automatically wrapped around the refactored file so it can be compiled to detect obvious errors. Compile errors are shown in a CodeJock tool panel at the bottom of the window, mirroring the Studio's behavior. The **Compare Code** action launches the configured comparison tool to inspect the refactoring's effect.

Unit testing

Unit testing is central to the new architecture. `DFUnit_TestRunner.exe` runs a suite of unit tests against `cRefactorFuncLib` to ensure that edits to the library do not break existing functionality. Tests are defined in `oUnit_Tests.pkg`; additional fixtures live under `AppSrc/Tests/`. The DFUnit framework was originally written by Ola Eldöy, enhanced by Bram NijenKamp, and adapted for this project.

DFRefactor 3.2.1.0

Dialog modernization

- All ModalPanel dialogs were converted to cRDCModalPanel.pkg. The new class automatically adds a status-panel row at the bottom of each dialog, makes all dialogs resizable, supports Status_Help consistently across all panels, and fixes a DataFlex bug in setting the Icon property. All anchors were adjusted to support resizing.

Status panel and tooltips

- The status-panel "No: xx of xx" message now reads "File: xx of xx" to clarify that files (not lines) are being counted.
- Status_Help is now derived automatically from psToolTip when no explicit Status_Help is provided.
- cRDCHeaderGroup.pkg was updated to display a small info bitmap at the end of the control to indicate that a tooltip is available. New icons and bitmap were added to support this.

Refactor view layout

- A new Standard Functions header group was added and RefactorView.vw was re-balanced.
- The Tab Size control was moved to the upper right, since it is used by all indentation logic — not just for If/Else/Begin constructs.
- Compound rows were changed to use explicit Begin/End blocks.

Other UI updates

- New Report.ico for the Report Functions group.
- New combo-form workspace selector with larger checkbox-slider icons.
- The "Run Now!" button's default state is now True.

Product title and registry

- The product title was changed from "Automated Code Refactoring for DataFlex" to "Automated Source Code Refactoring for DataFlex". Note that this changes the registry key under which workspaces are saved; existing history can be preserved by renaming the old registry key with Regedt32.

Open file selector

- An All Files option was added to the file selector.
- The path of the most recently opened file is now remembered, so re-opening a file from outside the AppSrc folder reopens at the same path.

Unused Variables logic (cRemoveUnusedLocals)

A series of issues was fixed (reported by Peter Bragg and others):

- Two-dimensional arrays are now checked for all basic DataFlex types.
- CS_ValidLeftCharacters was missing the = character; added.

- CS_ValidLeftCharacters extended to include the EOL character.
- A copy-paste error in the RetrieveFirstWord method (acting on sLine instead of sFirstWord) was corrected.
- End-of-method detection moved to a dedicated IsMethodEnd helper, improving robustness.
- Multi-line method declarations now strip parameters correctly (UnusedVarsTestMethodDeclareBrokenLine.pkg).
- Array declarations with a space between type and [] (String [] a) are now handled (UnusedVars-SpaceArrays.pkg).
- Variables with overlapping name prefixes (iSpace vs iSpaces) are now distinguished correctly (UnusedVars-PartialVariableNames.pkg).
- Variables whose names partly match a type keyword (e.g. sH was incorrectly absorbed into the type Short) are now removed correctly.

Tokenizer and editor

- New SciLexer.dll adds code-fold support for Type / End_Type blocks. This is used by the re-indent logic, which now indents Type blocks correctly instead of leaving them left-aligned.

Engine behavior

- Replaced the post-run Info_Box with the log-file dialog.
- Removed Sleep calls in favor of a WaitForFileToGetWritten message that waits for Windows to release a newly written file. Added the file's line count to the status panel so users can gauge progress on large files.

Bug fixes

- Fixed DfAbout.pkg: a commented-out line had broken the compile-date logic on DataFlex versions below 19.1.
- Fixed case-sensitivity bugs in ReplaceCalcToMoveStatement and IsKeywordInLine (reported by Michael Mullan).
- Fixed bug #131 — "CM images, do not remove blank lines".
- Fixed an issue with ChangeUClassToRefClass that emerged after running DFRefactor on its own source. Added safeguards for the "run on itself" scenario.
- Fixed cRDCSlideButton.pkg width clipping the right-hand text.

Library and licensing

- Library submodules are now pegged to fixed revisions so that upstream changes can no longer destabilize this code-base.
- The license was updated to note the artwork attribution.

Merges

- Merged Marco Kuipers's DfUnit tests.
- Merged Marco's regex-based implementation of ChangeInToContains. The regex path is currently disabled until a dedicated configuration dialog is implemented.

DFRefactor 3.1.0.0

Version-control consolidation

- Merged two SVN branches into a single branch (`DfRefactor`). The version number was bumped to 3.1 to reflect the consolidation.

New features

- **`cUnusedSourceFiles.pkg`** received a major update with significant performance improvements. The way source files are read into memory was optimized; the impact on large workspaces is substantial.
- **`UnusedSourceFiles.dg`** dialog (new). Lists source files that the **Unused source files** routine flagged as un-referenced. Files can be moved to the backup area directly from the dialog.
- **`LogFileDialog.dg`** dialog (new). Displays the log file produced by a refactoring run.
- **`cRDCSlideButton.pkg`** class (new). Replaces checkboxes on the Refactor view. In the Studio designer it appears as a checkbox; at runtime it renders as a "slide" toggle similar to those used in web applications.
- **Configurable selected-row color.** A new combo box in `ProgramSetup.dg` configures the selected-row background color used by `cRDCCJSelectionGrid` (`cCJGrid piSelectedRowBackColor`). The default — `clBlueGreyLight` — can be hard to read, so the color is now user-adjustable.

Select Folders improvements

- **Full subfolder paths.** The **Select Folders** grid now displays the full path for every folder *and* subfolder under the selected workspace, allowing subfolders to be individually included or excluded. This required changes throughout the file/folder collection routines.
- **Horizontal splitter.** A horizontal splitter was added between the Function List and the Select Folders grid so the grid can be resized to show more rows on larger displays. The original compact layout — to support low-resolution screens used by some testers — remains the default.
- **Vertical splitter** added for the lower part of the Refactor view.
- **Right-click context menu** on `cRDCCJSelectionGrid` with four standard actions (Toggle Current Item, Select All, Select None, Invert Selection) plus two optional actions (Add Folder, Remove Item From Grid) controlled by the properties `pbShowAddFolderMenuItem` and `pbShowRemoveFolderMenuItem`. The standalone **Add Folder** button was removed.
- **Cursor wait** while the Select Folders grid is being populated.

Bug fixes

- Fixed `DfAbout.pkg`: a commented-out line had broken the compile-date logic on DataFlex versions below 19.1.
- Fixed a folder/path-matching bug that manifested when a workspace path contained a `.` (for example `MyWorkspace2.0\AppSrc`). Replaced fragile `contains` checks with more robust path-comparison logic.

UI refinements

- Reindented `RefactorView.vw` by running DFRefactor on itself.
- Adjusted `cRDCSlideButton.pkg` width so it no longer clips text on the right.
- Removed all `cRDCHeaderGroup` frames after the splitter introduction left the screen too dense. The flatter look gives better visual hierarchy.
- Fixed color values for `clBlueGrey` and `clBlueGreyLight` (a purple value had been pasted in by mistake), which also corrected a handful of icon colors.
- `cRDCComboForm.pkg` drop-down list is now auto-sized to the widest text entry.
- Added keyboard shortcuts to the `cRDCCJSelectionGrid` right-click menu.
- `DFAbout` library set to "minimal" mode.
- Moved several `.ico` files to their respective library folders.
- A new toolbar button surfaces the **Unused Source Files** dialog.

DFRefactor 3.0.0.0

New features

- **Backup Files dialog.** Lists every backup file for the current workspace and supports moving selected files back to their original location with the **Move Files** button. Files are restored without date or existence checks.
- **`cRDCCJSelectionGrid` class.** A reusable grid class for displaying data and managing selection state. Used by the **Select Folders** grid on the Refactor view, the **Delete Workspace History** dialog, and the **Current Workspace Backup Files** dialog.
- **`cRDCHeaderGroup` tooltip support.** Tooltips can now be attached to header-group controls.

UI improvements

- **Balloon-style tooltips.** Changed throughout the application. Includes a new message, `DoChangeTooltipFormat`, on `ghoCJCommandbarSystem`.
- **Toolbar consolidation.** Workspace and source-file paths are no longer shown to the right of their controls; the path appears as a tooltip on hover. This allowed merging the first two toolbar rows into one.
- **Tab pages on the left.** The main view's tab pages were moved from the top of the window to the left side, giving more vertical screen space.
- **Configurable toolbar icon size.** A new setting in the **Program Settings** panel adjusts toolbar icon size.

- **Sortable Theme combo.** The Theme drop-down is now sorted.
- **Third function-list column.** Added to make better use of vertical screen real estate.
- **`cRDCCheckbox` info-bitmap click.** The "info" bitmap to the right of a checkbox now responds to clicks as if it were part of the checkbox object. A new property, `pbCreateInfoItem`, controls whether the info bitmap is created (`True` by default).
- **`cRDCHeaderGroup` text-size property.** A new property `pbUseLargeFontHeight` switches the header-group text between 10pt and 12pt.

Visual refresh

- New `ActionBackup.ico` icon. The previous icon implied a cloud backup, which was misleading.
- Grids now use `xtpReportThemeVisualStudio2012Light`, so selected-row checkboxes appear in bold.

Bug fixes

- Fixed a bug in `cRemoveUnusedLocals.pkg` (reported by Martin Arvidsson) when an end-of-line comment appeared on a variable declaration line.
- Fixed a related bug in `cRemoveUnusedLocals` where a comment on a variable declaration line — when no variables were used in the function — left the declaration in place, producing a compile error.
- Fixed a bug when a file was dropped onto `RefactorView.vw`.
- Fixed a bug with drag-and-drop on `RefactorView.vw`.

Internals

- The directory separator was extracted to a constant `CS_DirSeparator` and applied uniformly across the workspace.
- Reworked the inner processing loop: all involved folders are now collected first, then all files for those folders are gathered into a single struct array, and processing iterates over that array file-by-file. This replaced an accumulation of historical complexity.
- New `ProcessFile` logic: backup files are moved back to their original location if the source was unchanged after processing.

Documentation

- The `DFRefactor.chm` help file was updated with images reflecting the new look and feel.

What Is Refactoring?

Definition

Refactoring is a disciplined technique for restructuring existing code without changing its external behavior. Each refactoring is a small, behavior-preserving transformation: rename a variable, extract a method, replace a deprecated command with its modern equivalent. Applied consistently, refactorings keep a codebase readable, maintainable, and approachable to new contributors.

A change that alters behavior — replacing an algorithm with a faster one, fixing a bug, adding a feature — is not a refactoring. Behavior preservation is the defining property.

Inspiration: Martin Fowler's Refactoring

DFRefactor is inspired by Martin Fowler's book Refactoring: Improving the Design of Existing Code. The second edition (Addison-Wesley, 2019) updated the original 1999 catalogue of refactorings and codified the modern practice of small, behavior-preserving code transformations.

Fowler's central observation is that refactoring is not a one-time cleanup activity. It is a continuous discipline that runs alongside normal development. Done well, it keeps a codebase responsive to change and approachable to new contributors.

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."
— Martin Fowler

DFRefactor automates a subset of these refactorings for the DataFlex language. The functions it provides apply behavior-preserving transformations to legacy syntax, replacing deprecated commands with modern equivalents, removing unused declarations, and normalizing formatting and casing.

Why refactor DataFlex code?

The DataFlex language dates from the 1980s. Over four decades it has accumulated syntactic forms that newer versions have deprecated, retired, or hidden from the documentation. Code from character-mode DataFlex or the early Visual DataFlex era — Calc/MoveStr, [Found]/[FindErr] indicators, LT/EQ operators, command-style Pos/Replace, Shadow_State — usually still compiles in current Studio releases, but it is hard to read, hard to teach, and at odds with the idioms modern Studio tooling expects (statement completion, syntax coloring, the modern grid and list classes).

One boundary matters more than the rest: DataFlex 2021 (version 20.0) introduced native 64-bit and full Unicode support. Codebases whose roots predate that release tend to carry the most legacy syntax and gain the most from refactoring — Unicode in particular interacts with how source is read, written, and encoded. DFRefactor itself requires one of the two most recent DataFlex Studio versions installed to compile and run, but it can refactor source from any earlier version.

Bringing legacy DataFlex code up to modern standards has two main benefits:

Maintainability. Readable code is cheaper to maintain. Clear identifiers, consistent casing, single-purpose methods, and accurate comments reduce the time required to locate and correct defects. Refactoring also removes misleading or outdated comments, which are a recurring source of confusion.

Extensibility. Refactored code is easier to extend. Modern idioms and recognizable patterns make it possible to add new features without disturbing existing structure. Code that adheres to current Studio conventions can be edited more safely with Studio's own tooling.

Why automated refactoring?

Manual refactoring is reliable but slow. Across a large codebase, hand-applying even simple transformations — changing every Calc statement to Move, every Eq to =, every Current_Object to Self — is tedious and error-prone.

Automated refactoring:

- **Reduce manual effort**. A function that has been tested against many input shapes can be applied to thousands of lines in seconds.
- **Reduce error rates**. Refactoring functions in DFRefactor are exercised by unit tests in DFUnit_TestRunner before release.
- **Are reproducible**. Running the same set of functions over the same source produces the same result. This makes it possible to integrate refactoring into a CI process or to apply identical transformations across multiple branches.

DFRefactor is intentionally conservative: each function targets a narrow, well-defined transformation. Users are encouraged to apply a small number of functions per run and to commit (or back up) source code between runs. See Program Settings for the built-in backup mechanism.

Overview

DFRefactor is a refactoring tool for DataFlex source code. It is written entirely in DataFlex and is distributed as freeware under an MIT license.

Programs

DFRefactor ships as three executables:

- DFRefactor.exe — the main application. Most users only need this program. It selects refactoring functions, targets one file or a whole workspace, runs the transformations, and

produces a summary log. The program can also run unattended; see Command Line Usage.
- TestBench.exe — for advanced users. A harness for developing and testing new refactoring functions against a single source file. Refactored output can be compiled in-place to verify that the transformation produces valid DataFlex.
- DFUnit_TestRunner.exe — for advanced users. The unit-test runner. Exercises cRefactorFuncLib and its parent cBaseFuncLib against the fixtures in oUnit_Tests.pkg. Run after every change to the function library.

Operating modes

DFRefactor operates in one of two modes, selected from the toolbar:
- Workspace mode — applies the selected refactoring functions to every source file in the selected folders that matches the file-extension filter. Suitable for batch processing.
- File mode — applies the selected functions to a single source file. Suitable for targeted edits and for testing.

Typical workflow

1. Open a workspace by selecting a .sws file. A workspace is required even in File mode, because DFRefactor uses the workspace home folder as the root for its backup directory.
2. Choose Workspace or File mode in the toolbar.
3. In Workspace mode: choose source folders and the file-extension filter.
4. Select one or more refactoring functions from the function list.
5. Adjust per-function parameters in the Parameter column where applicable.
6. Open Program Settings to configure backup retention, comparison tools, and engine behavior.
7. Open Editor Settings to configure the Scintilla editor, including indent size and keyword casing used by editor-type functions.
8. Click Start Refactoring.

After completion, a summary log is displayed. The original files are preserved in a DFRefactor Backup folder inside the workspace home folder; see Program Settings for retention behavior. For a step-by-step walkthrough, see How to use the Program. For the internals of the refactoring engine, see How It All Works.

Editor view note

When one or more selected functions have type Editor — One File (for example EditorReIndent or EditorNormalizeCase), the Editor tab page is activated automatically at the start of the run. The Scintilla editor used by these functions is an OCX component that must be paged and visible to operate.

Getting Started

Reference pages for new users:
- How to use the Program — step-by-step walkthrough of a refactoring session.
- Toolbars — every toolbar button explained.
- Program Settings — backup retention, comparison tools, engine behavior.
- Editor Settings — Scintilla editor configuration (font, colors, keyword casing, indent size).

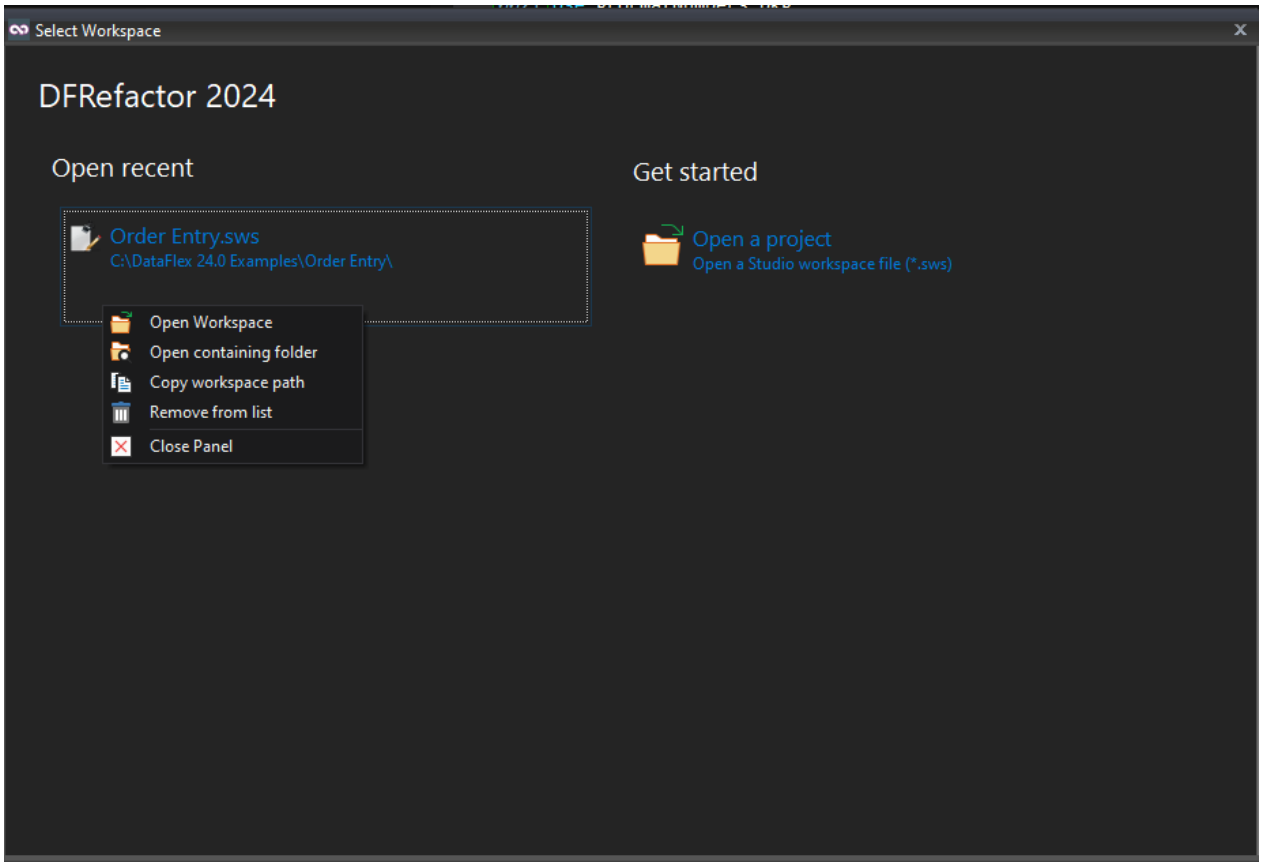
Background reading:
- What Is Refactoring — definition, motivation, why automation helps.
- Overview — the three DFRefactor programs and operating modes.
- How It All Works — internals of the refactoring engine, for developers extending the function library.
- Command Line Usage - run programs unattended.

How to use the Program

This page walks through a complete refactoring session.

1. Select a workspace

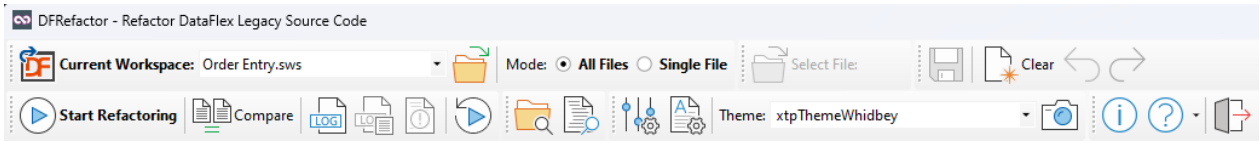
DFRefactor requires a workspace (.sws file). The workspace home folder is used as the root for the backup directory, even when refactoring a single file outside Workspace mode.



There are three ways to open a workspace:

- From the opening screen, click Get Started and select an .sws file, or pick a previously opened workspace from the Open Recent list.
- From inside DFRefactor, click the Open Workspace File button (folder icon) in the toolbar, or use the split-button dropdown to select a recent workspace.
- Drag an .sws file from Windows Explorer onto the application window.

Any .sws file from the workspace will do; the DataFlex version stamped on the file is not relevant. The Open Recent list holds up to 10 entries (FIFO); right-click an entry to remove it.



The full path to the selected workspace is shown in the status bar and as a tooltip when you hover over the workspace control. The toolbar control itself shows only the file name, to save space.

Selected Folders. The first time a workspace is opened, DFRefactor scans the home folder for sub-folders and populates the **Selected Folders** grid. Use the grid's right-click context menu to add, remove, or toggle the inclusion state of each folder. All changes are persisted to the database automatically.

2. Workspace or File mode

Select the operating mode in the toolbar. Alt+F toggles between the two.

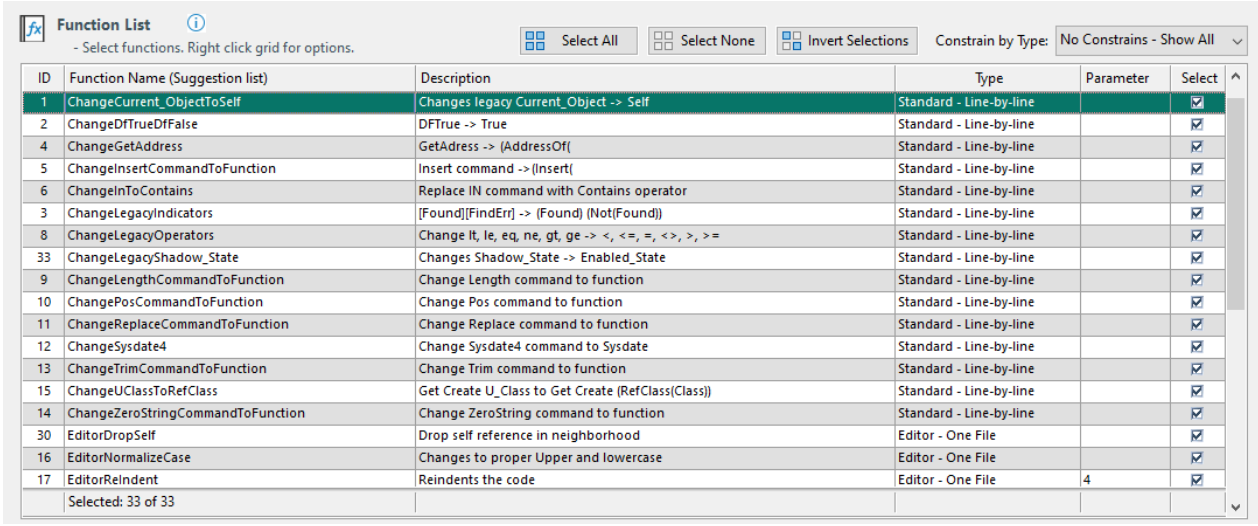
- **Workspace** mode applies the selected refactoring functions to every source file in the selected folders that matches the file-extension filter.

- **File** mode applies the selected functions to a single source file. To switch to File mode, click the Open Source File button above the mode radio buttons; this both opens the file and switches the mode.

3. Select refactoring functions

Tick the Select column for each function to include in the run. There are five categories of refactoring function:

Category	Effect
Standard	Modifies source code one line at a time according to the function's description.
Remove	Removes legacy syntax or unused declarations, one line at a time.
Editor	Reorganizes or reformats whole files via the Scintilla editor view.
Report	Reports findings without modifying source code.
Other	Operates on a whole file as a string array, or on the full file list.

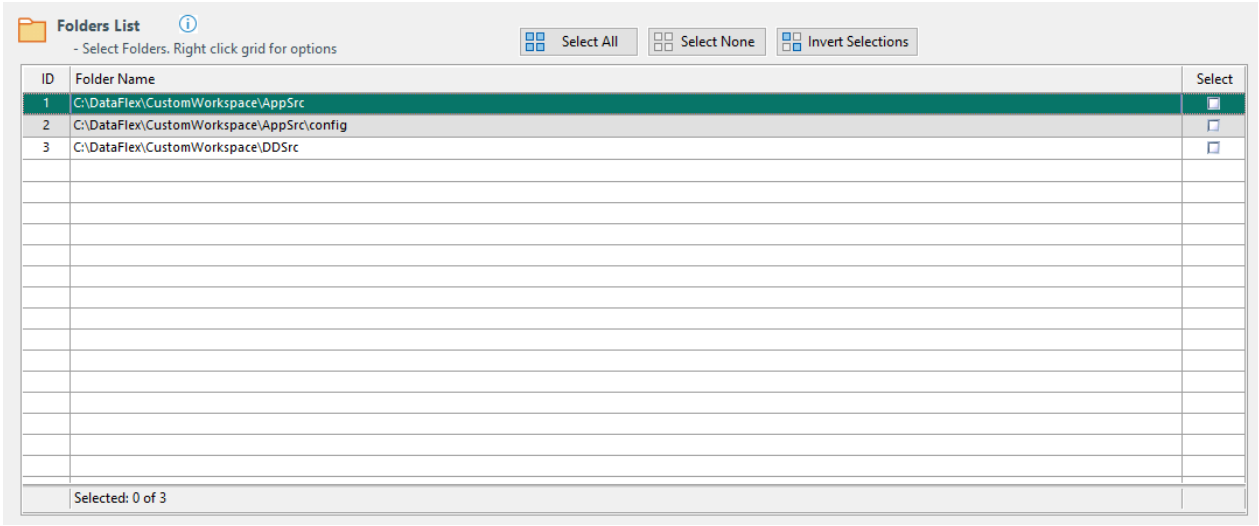


See Refactoring Functions for the per-function reference.

Grid controls. Use the selection buttons above the grid for bulk select/deselect, or the space bar to toggle a single row. The grid also has a right-click context menu.
The **Function Name** column is a search-suggestion list. Type any part of a function name to jump to matching rows.
Constrain Functions. The drop-down above the grid restricts the display to a single function type for easier browsing.
Type column. Indicates how data is passed to the function: one source line at a time (Standard, Remove), a full source file as a string array (Editor, Other — One File, Report — One File), or all selected files as a string array (Other — All Files, Report — All Files). See Refactoring Functions for details.
Parameter column. Some functions accept an additional sParameter. Double-click the Parameter cell to choose a value from the drop-down. Hover over the cell to read the help text for valid values.
Select column. Tick to include the function in the next run.

4. Source code folders (Workspace mode only)

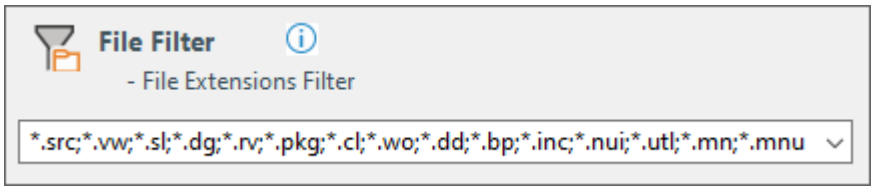
In Workspace mode, select the folders to include in the run. Use the space bar to toggle the selection state of a row, or the right-click context menu to add, remove, or invert selections. Drag-and-drop a folder from Windows Explorer to add it.



Note: The folder grid is disabled in File mode, because only one source file participates in the run.

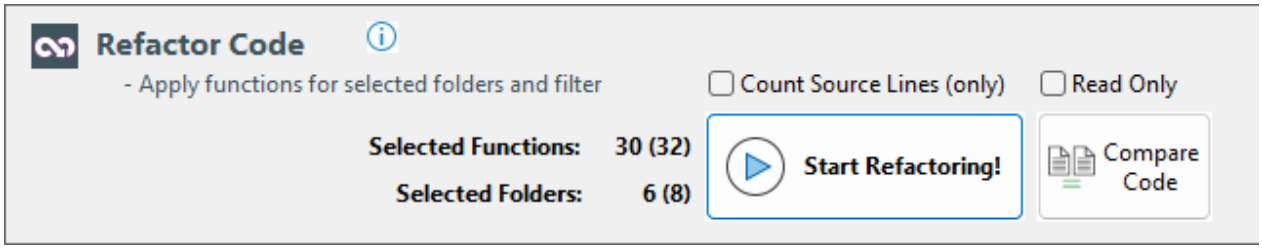
5. File extensions filter

Select a preset filter from the drop-down (for example *.pkg;*.vw;*.src) or enter your own wildcard combination. Press Ctrl+S to save a new combination as a preset.
Every source file under the selected folders (and their sub-folders) that matches one of the selected extensions will be included in the run.



6. Start refactoring

Important: Back up or check in your source code before running DFRefactor. *The built-in backup mechanism is a safety net, not a substitute for version control. The user is responsible for ensuring source code is safely preserved before any refactoring run. The author of this software makes no guarantee that the built-in backup mechanism always works as intended.*
The backup mechanism creates a DFRefactor Backup folder in the workspace home folder, with sub-folders mirroring the structure of the folders being processed.



Overwrite cycle. Re-running DFRefactor against the same workspace within the overwrite-cycle window (99 days by default) does not overwrite the existing backup. This preserves the original source for the workspace through a series of incremental refactoring runs. After the overwrite-cycle window expires, the next run creates fresh backups. The cycle length is configurable in **Program Settings**.
Click the **Start Refactoring** button in the lower right corner of the Refactoring view to begin. Large

workspaces may take several minutes.
Run-time options

- **Count Source Lines (only)**. No refactoring functions are applied. Instead, DFRefactor counts non-blank, non-comment source lines across all files matching the current filter and folder selection. Studio-generated ActiveX/OCX code is excluded from the count.
- **Read Only**. Selected functions are invoked, and the summary log reports how many changes would have been made — but no source files are modified. Useful for estimating the scope of a change before committing to it.
- **Compare Code.** If a comparison tool has been configured in Program Settings, this button launches it after the run to show the differences between original and refactored source.

7. Compare code

If a comparison tool is configured in Program Settings, the Compare Code button opens it after a run, showing the differences between the backup (original) and the working (refactored) source.

Command-Line Usage

`DFRefactor.exe` and `DFUnit_TestRunner.exe` both support unattended operation from the command line. This enables scripted batch refactoring and continuous-integration use, where no user interaction is possible.

DFRefactor command-line parameters

Syntax rules

- A flag and its value form a pair separated by a space (for example `/sws "C:\Proj\My.sws"`).
- Parameters may be supplied in any order.
- Paths must be enclosed in single or double quotes.
- Flag spelling is case-insensitive.
- Each flag must be prefixed with a forward slash (`/`) or a dash (`-`).
- Only the parameters relevant to the run need to be supplied.

Flags

Flag(s)	Value	Meaning
`/h`, `/help`, `/?`, `?`	—	Display the parameter help, then exit. Takes precedence over `/debug`.
`/debug`	—	Echo every parameter that was received, then exit without refactoring.
`/sws`	`"path.sws"`	The workspace file. Mandatory unless `/c` is used.
`/f`, `/file`	`"path"`	A single source file (full path). Takes precedence over `/sws`.
`/d`, `/dirs`, `/folders`	`A;B;C`	A semicolon-separated list of folder names. These are **added to** the folders already saved for the workspace, not a replacement. Use together with `/sws`.
`/b`, `/batch`	—	Start and run the refactoring session automatically and silently. The UI is visible

Flag(s)	Value	Meaning
		while the run proceeds, but no interaction is required; actions and errors are logged, not shown.
/c, -c, --console	"path.ini"	Use an ini-file instead of /f / /d. Implies /batch (it is always treated as on).

Precedence

- A single source file (/f) takes precedence over a workspace (/sws).
- An ini-file (/c) forces batch mode on, regardless of whether /batch was supplied.
- If /batch is omitted (and no /c), the program starts and displays the supplied parameters but does **not** begin refactoring automatically — the user must start the run.

Examples

Run all previously selected functions across two folders of a workspace, silently:
```
DFRefactor.exe /sws "C:\DataFlex 24.0 Examples\Order Entry\Order Entry.sws" /d AppSrc;DDSrc /batch
```
Run a single file:
```
DFRefactor.exe /f "C:\Proj\AppSrc\Customer.vw" /batch
```
Run from a saved ini-file (recommended — see below):
```
DFRefactor.exe /c "DFRefactorCmdLineWS.ini"
```

Using an ini-file

Instead of specifying the workspace, folders, and file filter as individual flags, the entire configuration can be supplied as an ini-file via /c. The default ini-file name is DFRefactorCmdLine.ini; the default location is the workspace Home folder. If the file is elsewhere, supply its full path.
The ini-file has five sections:

```
[SWS File Section]
SWS File Name=C:\Proj\My.sws

[User Name Section]
User Name=rdcto

[Folder Section]
Folder Name1=C:\Proj\AppSrc
Folder Name2=C:\Proj\DDSrc

[File Filter Section]
File Filters=*.src;*.vw;*.sl;*.dg;*.rv;*.pkg;*.cl;*.wo;*.dd;*.bp;*.inc;*.nui;*.utl;*.mn;*.mnu

[Functions Section]
Function Name1=ChangeCalcToMoveStatement
Function Name2=ChangeCurrent_ObjectToSelf
Function Name3=ChangeLegacyOperators
```

Notes:

- [Folder Section] entries are numbered (Folder Name1, Folder Name2, …) and are **added to** the folders already saved for the workspace.
- [Functions Section] entries are numbered (Function Name1, …) and must use the exact function names shown on the Refactoring Functions pages.
- [File Filter Section] uses a single semicolon-separated File Filters value.

Exporting settings to an ini-file

Writing an ini-file by hand is error-prone. Instead, configure the workspace, folders, file filter, and function selection interactively, then use the **Export to ini-file** icon next to the "Undo Refactoring" icon . This writes a ready-to-use ini-file reflecting the current configuration, which can then be passed to /c on the command line and committed to source control for repeatable runs.

Batch-mode behavior

When a run is started with `/batch` (or implicitly via `/c`):

- The program UI is shown on screen, but no user interaction is required or expected.
- All actions, and any errors, are written to the log files rather than displayed in dialogs. See the **Refactor Engine Error Log** and **Other Log files** toolbar buttons described in Toolbars.
- The program exits automatically when the run completes.

Running the unit-test runner headless

`DFUnit_TestRunner.exe` supports a console mode for automated and CI use:
`DFUnit_TestRunner.exe -c`

- `-c` or `--console` runs the full test suite unattended, writes the results to the console, and exits the process with code `0` when all tests pass or `-1` when any test fails.
- In console mode the runner never returns to a UI; the process terminates as soon as the suite finishes.
- Without `-c` / `--console`, the runner starts in interactive UI mode.

Caution — overlapping flag names. *`-c` and `--console` mean different things in the two programs: for `DFRefactor.exe` they select a configuration ini-file and require a path argument; for `DFUnit_TestRunner.exe` they select console / CI mode and take no argument. They are separate programs with independent argument handling.*

Example: continuous-integration pipeline

A typical CI sequence:

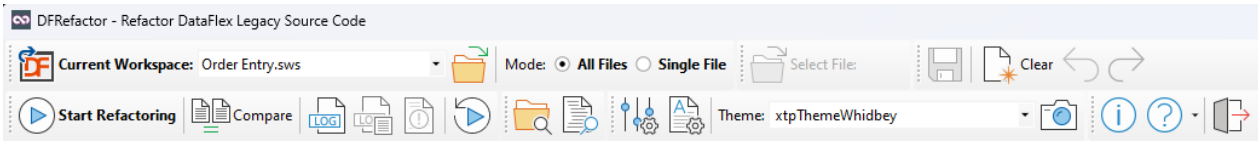
1. Once, interactively: configure the workspace and function selection, then **Program Settings** → **Export to ini-file** to produce (for example) `DFRefactorCmdLineCI.ini`. Commit that ini-file to source control.
2. In the CI job, run the refactoring:
   `DFRefactor.exe /c "DFRefactorCmdLineCI.ini"`
3. Run the unit tests and gate the build on the exit code:
   `DFUnit_TestRunner.exe -c`
   A non-zero exit code (`-1`) fails the build.

See also: How to use the Program, Program Settings, Overview.

Toolbars

DFRefactor's toolbars are organized into groups by function. Each button is described below.

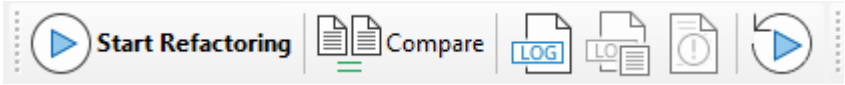
Workspace and file selection



Open Workspace / Current Workspace (Alt+W). Click the folder icon to select an .sws workspace file. The split-button dropdown lists previously opened workspaces. The control shows only the file name; hover for the full path.
Refactor Mode (Alt+F). Toggle between **File** and **Workspace** modes. In File mode, DFRefactor operates on a single source file. In **Workspace** mode, it operates on every source file under the selected folders that matches the file-extension filter.

Save Changes (Ctrl+S or F2). Saves changes made in the **Editor** view.
Clear (Ctrl+F5). Clears the current workspace and file selection.
Undo / Redo Editor Changes (Ctrl+Z / Ctrl+Y). Undo or redo the most recent change inside the Editor view.

Run controls



Start Refactoring. Runs the selected refactoring functions against the selected folders and file-extension filter (Workspace mode), or against the open source file (File mode).
Compare Tool. **Launches the external file-comparison tool, if one has been configured in Program** Settings. Use it to compare the backup (original) and the refactored source after a run.
Show Log File (Alt+L). Opens the most recent run summary, saved as DFRefactoringLogfile.txt in the workspace's DFRefactor Backup sub-folder.
Other Log Files. Some refactoring functions produce their own log files in addition to the main summary. Examples include ReportUnusedSourceFiles and ReportUnusedUseStatements. This button surfaces those secondary logs when they exist.
Refactor Engine Error Log. The engine runs inside a BusinessProcess, which suppresses errors so a single failure does not interrupt a long run. Any errors that occurred are written to the StatLog table and surfaced here.
Undo Refactoring (Alt+U). Opens the undo dialog. The dialog lists files in the DFRefactor Backup sub-folder structure and lets you move them back to their original locations, overwriting the refactored versions.
Open Containing Folder. Opens Windows Explorer on the workspace home folder. If a source file is open, the file is highlighted in Explorer.

External tools

StarZen's DataFlex Source Explorer. Launches the StarZen Source Explorer if it has been configured in **Program Settings**. Like the compare tool, it must be downloaded and installed separately.

Settings

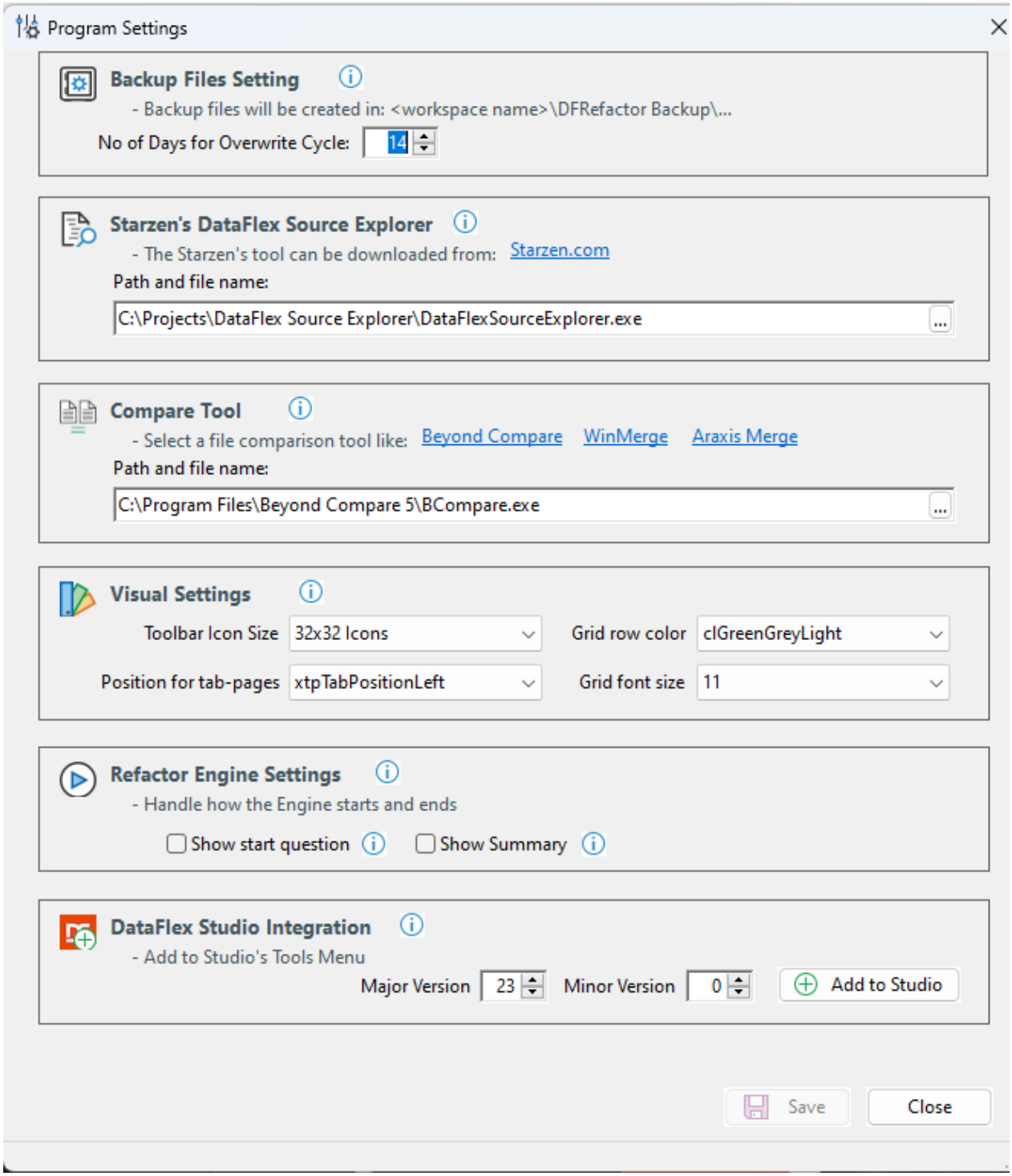
Program Settings. Opens the **Program Settings** dialog (backup retention, comparison tool, theme, engine behavior).
Editor Settings. Opens the **Editor Settings** dialog (Scintilla editor: colors, font, keyword lists, keyboard shortcuts, indent size).

Help and exit

About. Opens the About dialog, showing version information and credits.
Help. Opens this help file. An online version may be more current than the local CHM.
Exit. Closes the program.

Program Settings

The **Program Settings** dialog (opened from the toolbar) controls backup retention, external tool integrations, visual preferences, and engine behavior.



Hover the mouse over the round-ring information symbols next to each control to read additional context.

Backup Files

Backup files are written to `<workspace home>\DFRefactor Backup\...`, mirroring the source folder structure.

Number of days before overwriting backup files. This controls the overwrite cycle. Within the cycle, the *original* source file is preserved across multiple refactoring runs — DFRefactor refuses to overwrite an existing backup. This protects the original from being lost when the user applies a series of incremental refactoring functions over several days. After the cycle expires, the next run creates fresh backups, overwriting whatever was previously kept.

The cycle defaults to 99 days. Adjust it to match the rhythm of your refactoring sessions.

> ***The user is solely responsible for backing up source code before running DFRefactor.*** *The built-in backup mechanism is a convenience, not a guarantee. Always check source code into*

version control or take an independent backup before starting a refactoring session.

StarZen's DataFlex Source Explorer

A third-party tool, downloaded and installed separately. The settings page provides a link to the download. Once configured, the tool can be launched from the DFRefactor toolbar to examine and analyze source code.

Compare Tool

A third-party file-comparison tool, downloaded and installed separately. Three suggestions are provided, each with a download link. Once configured, the **Compare Code** toolbar button opens the comparison tool to show the differences between original and refactored source after a run.

Visual Settings

Visual preferences for the DFRefactor UI. If the source code to DFRefactor is available, an additional setting becomes visible. Uncomment the following line near the top of `DFRefactor.src` to enable a theme combo box in the toolbar:

```
//Define CI_ShowBlingStuff
```

Themes change the color palette and accent colors of the application. The grid font size in this section does not apply to drop-down selection lists.

Refactor Engine Settings

Controls the interactive checkpoints around a refactoring run:

- **Show start question.** When enabled, an `info_box` is displayed before each run, summarizing the number of selected functions and folders and reminding the user to check in source code first. When disabled, the run starts immediately when **Start Refactoring** is clicked.
- **Show log file dialog after run.** When enabled, the log-file dialog is opened automatically after a run completes. When disabled, the run finishes silently; the log file can still be viewed via the **View Logfile** toolbar button at any time.

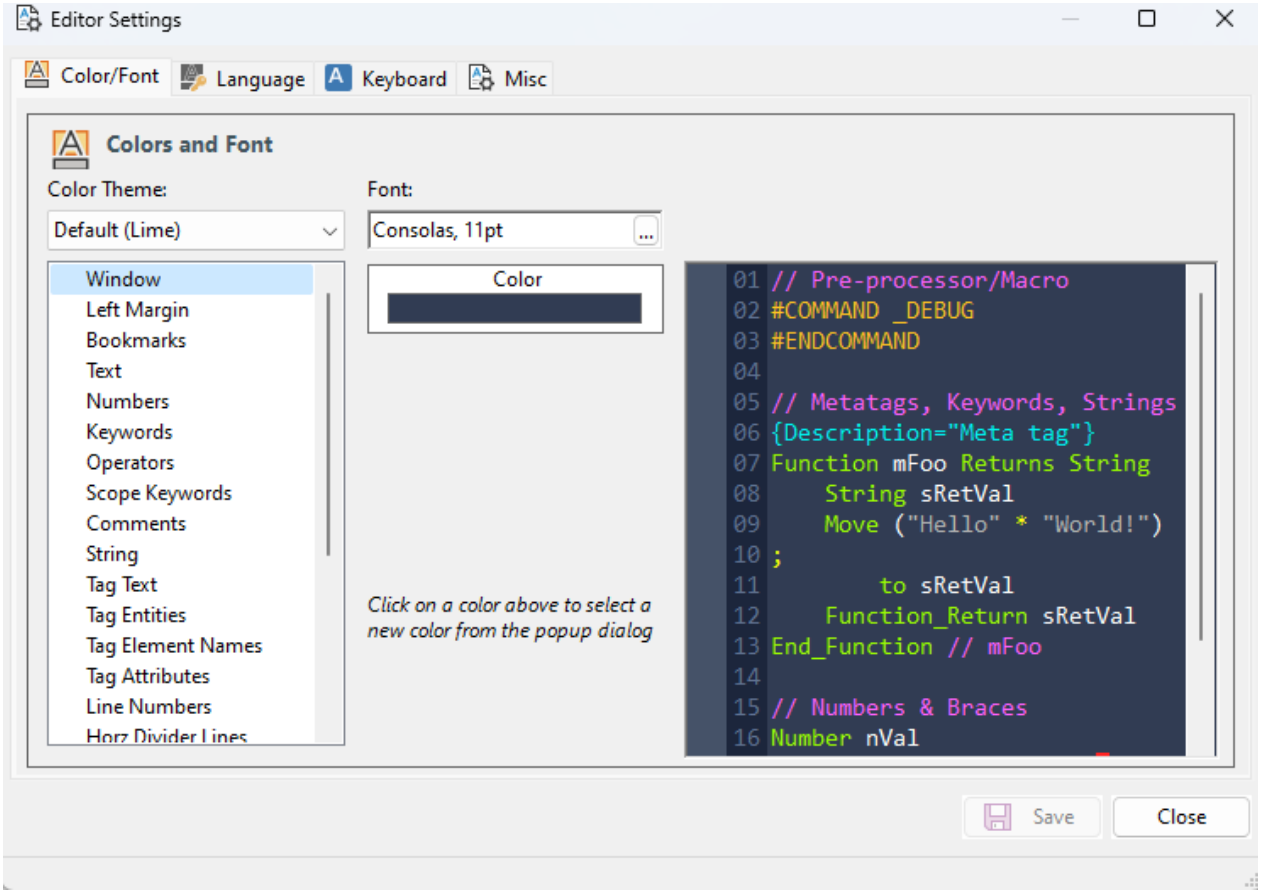
Leaving both checkboxes enabled is recommended.

DataFlex Studio Integration

Adds DFRefactor to the **Tools** menu of DataFlex Studio for quick access. The integration can target a different Studio version than the one used to compile DFRefactor; for example, if DFRefactor is compiled with DataFlex 2024, it can still be integrated into DataFlex Studio 20.0.

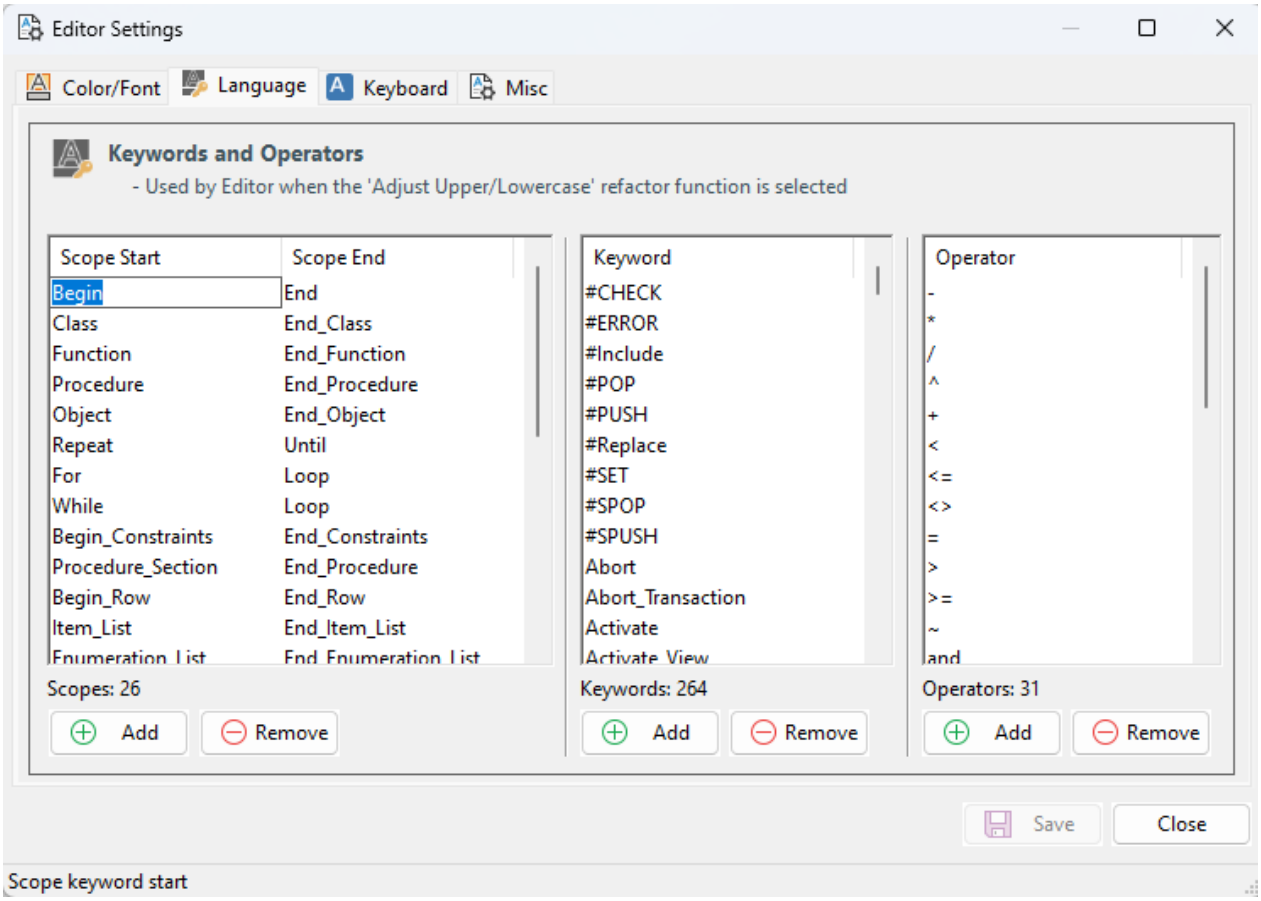
Editor Settings

The Editor Settings dialog (opened from the toolbar, Alt+D) configures the Scintilla editor used by the Editor view. The Scintilla editor is what Editor-type refactoring functions (EditorReIndent, EditorNormalizeCase) operate against, so these settings have a direct effect on those functions' output.



Colors and Font

Choose a color theme for the editor, or set individual colors per syntax element. Click the ... button next to the **Font** control to select a font.

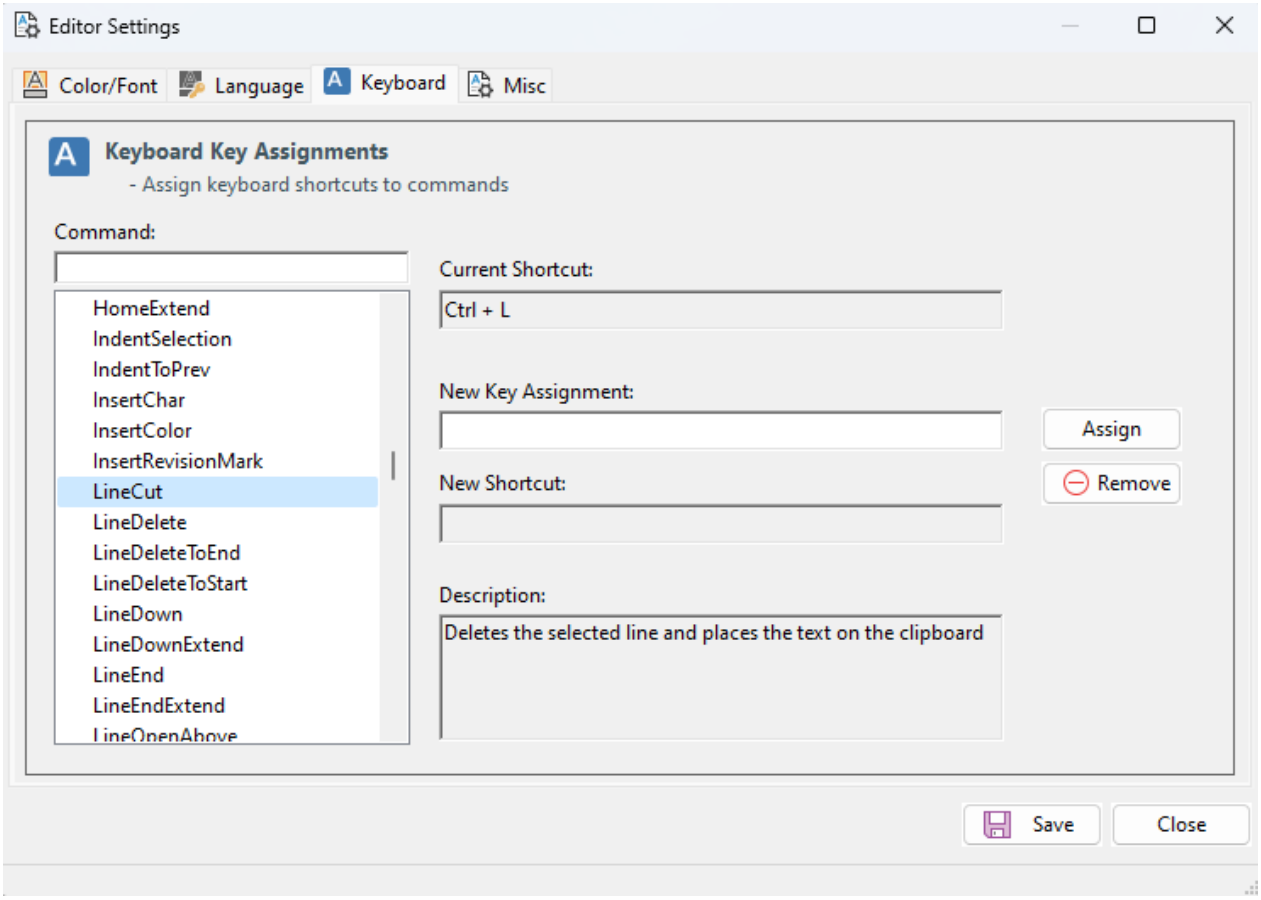


Keywords and Operators

Three editable lists:

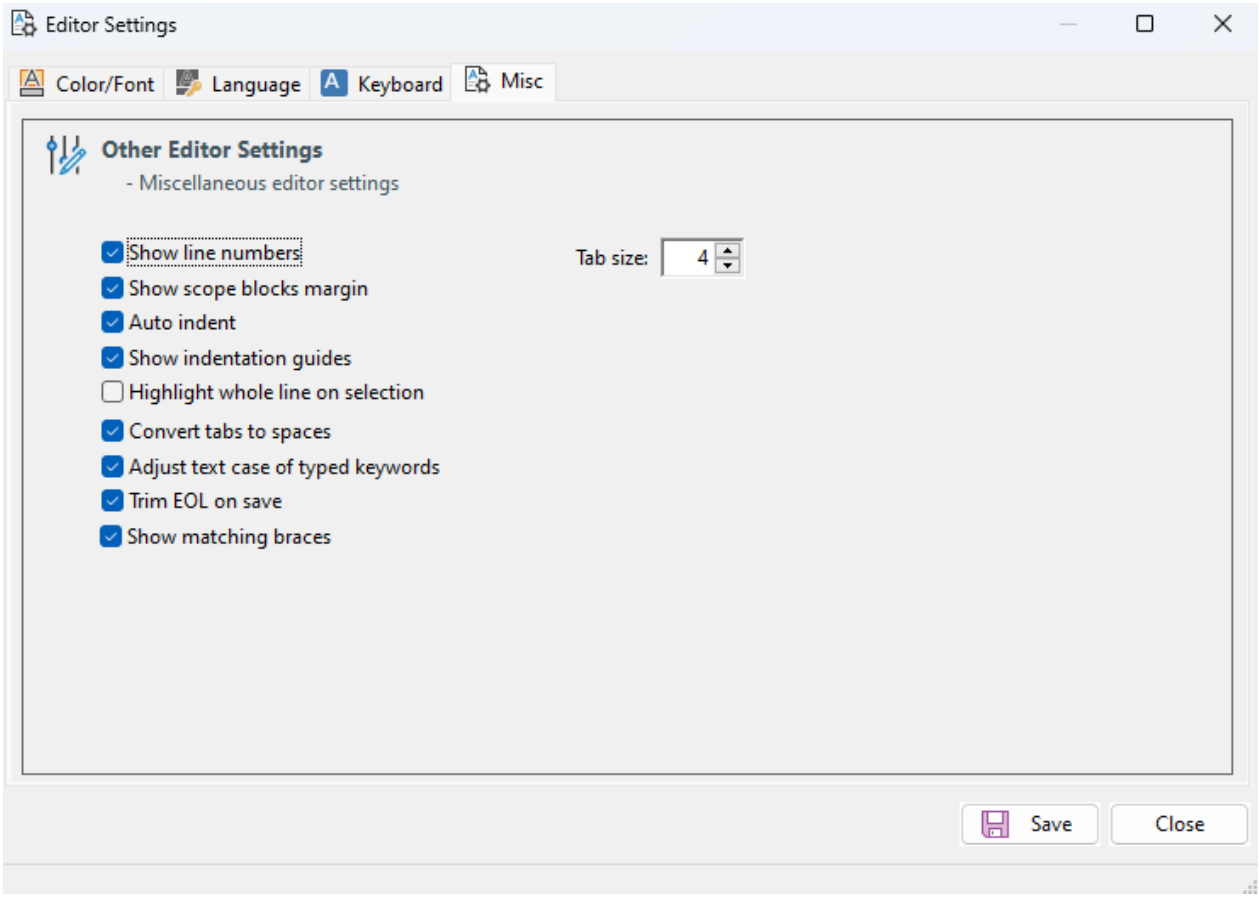
- **Scoped Words** (Start/End pairs such as Begin/End, Object/End_Object)
- **Keywords**
- **Operators**

These values are read at program startup and held in string-array properties used by the refactoring engine. Editing them has a profound effect on the refactoring outcome, because tokenization and keyword normalization depend on these lists.
Caution: change these lists only when you understand the consequences *. A typical reason to edit is to* ***add a custom command to the Keyword list*** *so that it is treated as a keyword during refactoring (for example, normalized to the casing used elsewhere in your codebase).*



Keyboard Key Assignments

Configures keyboard shortcuts for the Scintilla editor. The Scintilla editor used by DFRefactor is independent of DataFlex Studio's CodeMax editor, so its bindings are managed separately. Use this dialog to map shortcuts to match (as closely as practical) the bindings you use in Studio.



Other Editor Settings

Miscellaneous editor preferences. Hover over each checkbox for an explanation.
Tab Size. The number of spaces used for each indent level. This setting is important for EditorReIndent, which uses it to determine indentation when reflowing code. The same setting is exposed on the **Function List** grid — find the EditorReIndent row, double-click its **Parameter** column, and choose a tab size between 1 and 8 (default: 4).

Refactoring Functions

DFRefactor's refactoring functions are grouped by **method type**, which determines what data the engine passes to each function and how often it is called.

Operating modes

The selected refactoring functions are applied either to a single file or to the whole workspace:

- **File mode** applies the selected functions to one source file (the one currently open).
- **Workspace mode** applies the selected functions to every source file under the selected folders that matches the **File Filter** drop-down. Folders are selected in the **Source Code Folders** grid.

Function categories

Each category corresponds to one or more `MethodType` values declared in the function's `{ Description }` meta-tag. The engine uses this to decide what to pass:

Standard — Line by Line
Modifies source code one line at a time. The engine calls each selected function for every line of every selected file, with the line passed by reference. The function returns `True` if the line was changed. `MethodType = eStandardFunction`.

Remove — Line by Line
Like Standard, but specifically targets lines or constructs to be removed. The function returns `True` if the line should be removed. `MethodType = eRemoveFunction`.

Editor — One File
Operates on a whole file via the Scintilla editor view. Used for transformations that need access to

the editor's tokenization (re-indentation, keyword case normalization). The Editor view is paged automatically when an Editor function runs. `MethodType = eEditorFunction`.

Report — One File
Reports findings against a single file; makes no source changes. `MethodType = eReportFunction`.
No functions of this type are currently shipped.

Report — All Files
Reports findings against all selected files at once; makes no source changes. `MethodType = eReportFunctionAll`.

Other — One File
Operates on a whole file passed as a string array. Used when a transformation needs visibility across lines (removing consecutive blank lines, finding unused local variables). `MethodType = eOtherFunction`.

Other — All Files
Operates on all selected files at once, receiving a string array of file paths. Used for cross-file transformations such as renaming data-dictionary objects consistently. `MethodType = eOtherFunctionAll`.

Commercial services

For commercial development of custom refactoring functions or full legacy-code migration projects, see Feedback.

Standard — Line by Line

Functions of type `eStandardFunction` operate on one source line at a time. The engine calls each selected Standard function for every line of every selected file. The line is passed by reference; the function returns `True` if the line was modified.
Functions are listed alphabetically.

ChangeCalcToMoveStatement

Replaces the legacy `Calc`, `MoveInt`, `MoveReal`, `MoveNum`, and `MoveStr` commands with the modern `Move` command. The legacy commands still compile, but Studio's statement-completion does not work with them.

Before:
```
Calc(iVar + 2) to iMyVar
MoveStr "New Label" to sMyVar
```
After:
```
Move (iVar + 2) to iMyVar
Move "New Label" to sMyVar
```

ChangeCurrent_ObjectToSelf

Replaces the legacy keyword `Current_Object` with `Self`.

Before:
```
Procedure RunReport
    Send RunReport to (oMainReport(Current_Object))
End_Procedure
```
After:
```
Procedure RunReport
    Send RunReport to (oMainReport(Self))
End_Procedure
```

ChangeDfTrueDfFalse

Replaces the legacy constants `DFTrue` and `DFFalse` with `True` and `False` respectively.

ChangeGetAddress

Replaces the legacy `GetAddress` command with the `AddressOf()` function.

Before:
```
GetAddress of sVal to aAddress
```
After:
```
Move (AddressOf(sVal)) to aAddress
```

ChangeIfNotCommandToExpression

Converts the legacy `IFNOT` command into an `If (Not(...))` expression.

Before:
```
IFNOT Found Begin
```
After:
```
If (Not(Found)) Begin
```

ChangeInsertCommandToFunction

Replaces the legacy `Insert` command with the `Insert()` function.

Before:
```
Insert "," In sText At 2
If sOne Eq "A" Insert "B" in sOne at 2
```
After:
```
Move (Insert(",", sText, 2)) to sText
If sOne Eq "A" Move (Insert("B", sOne, 2)) to sOne
```

ChangeInToContains

Replaces the legacy `In` operator with the `Contains` operator. The function swaps the operand order and adds parentheses.

Before:
```
String sSub sAlphabet
Move "abcdefghijklmnopqrstuvwxyz" to sAlphabet
Move "abc" to sSub
If sSub in sAlphabet Send Info_Box sSub "Part of Alphabet"
```
After:
```
String sSub sAlphabet
Move "abcdefghijklmnopqrstuvwxyz" to sAlphabet
Move "abc" to sSub
If (sAlphabet Contains sSub) Begin
    Send Info_Box sSub "Part Of Alphabet"
End
```

ChangeIndexNumber

Changes index-number references in Find / vFind / Constrained_Find /
Constrained_Clear / For_all commands from one index number to another,
SCOPED to a specific table. Useful after a table-index restructure
that renumbers indexes for **one** table.
Rewrites BOTH `Index.<N>` and bare-integer forms in these positions.

DDO form recognized: o<TableName>_DD and
o<TableName>_DataDictionary (case-insensitive).
Other DDO naming conventions need to be handled manually.

Not touched: Find EQ <table>.<field> (constant form), "-1", "recnum"

Parameter (`sParameter`): a comma-separated list of three parameters — `MyTableName,FromIndexText,ToIndexText`. Default: `None (if none specified, the function does nothing)`.

When the parameters column value of the "Select Functions" view grid for the "ChangeIndexNumber" row, has been changed to:
"MyTable, 1, 2"

Before:
```
Send Find of oMyTable_DD EQ 1
Send Find of oMyTable_DD EQ Index.1
vFind MyTable.File_Number 1 GT
Constrained_Find MyTable GT by Index.1
Constrained_Clear eq Table name by 1
For_All order by 1
```

After:
```
Send Find of oMyTable_DD EQ 2
Send Find of oMyTable_DD EQ Index.2
vFind MyTable.File_Number 2 GT
Constrained_Find MyTable GT by Index.2
Constrained_Clear eq Table name by 2
For_All order by 2
```

ChangeLeftCommandToFunction

Replaces the legacy `Left` command with the `Left()` function.

Before:
```
Left sVar 5 to sLeft
```
After:
```
Move (Left(sVar, 5)) to sLeft
```

ChangeLegacyIndicators

Converts square-bracket indicator expressions to function-call expressions using `Found` / `Not(Found)` / `FindErr` / `Not(FindErr)`.

Note: there is a related function, `ChangeSquareBracketsIndicators`, with overlapping behavior. Test both against a sample of your code to determine which produces better results for your code patterns.

Known limitations:

- Handles at most two booleans within a single bracket pair (e.g. `[Found Select]`).
- Does not handle `GROUP` or `ALL` indicator directives. When either appears, the line is left unchanged.

Examples:
```
[Select] Indicate Select as Windowindex Eq Fieldindex   →  Move (WindowIndex = FieldIndex) to Select

[Found] Command                                         →  If Found Command
While [Not Found]                                       →  While (Not(Found))
[Found|Not Found|FindErr|Not FindErr] While             →  While (Not(Found))
[Found] Indicate Found as Invoice.CustNum eq Customer.Number
    →  If (Found) Move (Invoice.CustNum eq Customer.Number) to Found
If [Not Found] Reread hTable                            →  If (Not(Found)) Reread hTable
[~Found] begin                                          →  If (Not(Found)) Begin
[Found ~Found] Begin                                    →  If (Found and Not(Found)) Begin
```

ChangeLegacyOperators

Replaces the legacy comparison operators with their symbolic equivalents:

Legacy	Replacement
LT	<
LE	<=
EQ	=
NE	<>
GT	>

Legacy	Replacement
GE	>=

Operators inside `find` operations are not modified.

Before:
```
Integer iA iB
If (iA Eq iB) Begin
    Procedure_Return
End
```
After:
```
Integer iA iB
If (iA = iB) Begin
    Procedure_Return
End
```

ChangeLegacyShadow_State

Replaces `Shadow_State` and `Object_Shadow_State` with `Enabled_State`. The boolean is inverted, since `Shadow_State = True` means *disabled* whereas `Enabled_State = True` means *enabled*.

Before:
```
Set Shadow_State [of oObject] to True
Set Object_Shadow_State [of oObject] to True
```
After:
```
Set Enabled_State [of oObject] to False
Set Enabled_State [of oObject] to False
```

ChangeLengthCommandToFunction

Replaces the legacy `Length` command with the `Length()` function.

Before:
```
Length sVar to iLength
```
After:
```
Move (Length(sVar)) to iLength
```

ChangePosCommandToFunction

Replaces the legacy `Pos` command with the `Pos()` function.

Before:
```
Pos "def" in sWhole to iPos
```
After:
```
Move (Pos("def", sWhole)) to iPos
```

ChangeReplaceCommandToFunction

Replaces the legacy `Replace` command with the `Replace()` function.

Before:
```
Replace "two weeks" in sSentence with "one week"
```
After:
```
Move (Replace("two weeks", sSentence, "one week")) to sSentence
```

ChangeRightCommandToFunction

Replaces the legacy `Right` command with the `Right()` function.

Before:
```
Right sVar 5 to sRight
```
After:
```
Move (Right(sVar, 5)) to sRight
```

ChangeSquareBracketsIndicators

Converts square-bracket indicator expressions to parenthesized expressions.

Note: there is a related function, `ChangeLegacyIndicators`, with overlapping behavior. Test both against a sample of your code to determine which produces better results for your code patterns.

Known limitation: does not handle `GROUP` or `ALL` indicator directives. When either appears, the line is left unchanged.

Examples:

```
While [Not Found]                          →  While (Not(Found))
[Not Found] While                          →  While (Not(Found))
If [Not Found] Reread hTable               →  If (Not(Found)) Reread hTable
[~Found] Begin                             →  If (Not(Found)) Begin
[Found ~Found] Begin                       →  If (Found and Not(Found)) Begin
```

ChangeSysdate4

Replaces the obsolete `Sysdate4` command with `Sysdate`.

Before:
```
Sysdate4 dDate
```
After:
```
Sysdate dDate
```

ChangeTrimCommandToFunction

Replaces the legacy `Trim` command with the `Trim()` function.

Before:
```
Trim sVal to sVal
```
After:
```
Move (Trim(sVal)) to sVal
```

ChangeUClassToRefClass

Replaces the legacy `Get Create U_ClassName` pattern with `Get Create (RefClass(ClassName))`.

Before:
```
Get Create U_Array to hoArray
```
After:
```
Get Create (RefClass(Array)) to hoArray
```

ChangeZeroStringCommandToFunction

Replaces the legacy `ZeroString` command with the `ZeroString()` function.

Before:
```
ZeroString iLength to sParameter
```
After:
```
Move (ZeroString(iLength)) to sParameter
```

NormalizeArrayNotation

Removes the space between an array type name and the `[]` brackets in variable declarations.

Before:
```
String [] asAddress
Integer [] aiCounts
```
After:
```
String[] asAddress
Integer[] aiCounts
```

RemoveEndComments

Removes end-of-line comments after `End_Class`, `End_Object`, `End_Function`, and `End_Procedure`.

Before:
```
Procedure MyNewProcedure String sText
    Send Update
End_Procedure //MyProcedure
```
After:
```
Procedure MyNewProcedure String sText
    Send Update
End_Procedure
```

RemoveLocalKeyWord

Removes the legacy `Local` keyword from variable declarations inside functions and procedures.

Before:
```
Procedure Activate Returns Integer
    Local Integer iRetval iOk
```
After:
```
Procedure Activate Returns Integer
    Integer iRetval iOk
```

RemovePropertyPrivate

Removes the `Private` keyword from `Property` declarations.
Note: in rare cases removing `Private` exposes a property that was deliberately hidden. Review removal sites and re-add `Private` selectively if needed.

Before:
```
Property String pSelStart1 Private ""
Property String pSelStop1  Private ""
```
After:
```
Property String pSelStart1 ""
Property String pSelStop1  ""
```

RemovePropertyPublic

Removes the `Public` keyword from `Property` declarations. `Public` is the default visibility for properties; explicit `Public` adds no information.

Before:
```
Property String pSelStart1 Public ""
Property String pSelStop1  Public ""
```
After:
```
Property String pSelStart1 ""
Property String pSelStop1  ""
```

RemoveTrailingSpaces

Removes trailing whitespace (spaces and tabs) at the end of each line.

SplitInlineIfElseLine

Splits single-line `If`/`Else` statements into multi-line constructs. Does *not* split if the trailing keyword is `Break`, since doing so would change the program's control flow.
Parameter (`sParameter`): the split style. Valid values:

Value	Effect
`eSplitBySpaceAndSemicolon`	Line break with a space followed by `;`
`eSplitBySemicolon`	Line break with no space followed by `;`
`eSplitToBeginEndBlock`	Line break with a `Begin` / `End` block (default)

The indent size of the new block is taken from the **Tab Size** setting in Editor Settings (also exposed on the `EditorReIndent` row's **Parameter** column).

Source:
```
If (iRetval = 0) Procedure_Return
```
With `eSplitBySemicolon`:
```
If (iRetval = 0);
    Procedure_Return
```
With `eSplitBySpaceAndSemicolon`:
```
If (iRetval = 0) ;
    Procedure_Return
```
With `eSplitToBeginEndBlock`:
```
If (iRetval = 0) Begin
    Procedure_Return
End
```

Remove — Line by Line

Functions of type `eRemoveFunction` operate one source line at a time. They are used to remove legacy or generated lines from source code. The function returns `True` when a line should be removed.

RemoveOldStudioMarkers

Removes legacy `//AB` Studio IDE markers (`//AB/`, `//AB-StoreStart`, `//AB-StoreEnd`, etc.). Returns `True` when at least one marker is found on the line.

Before:
```
//AB/ Object    // prj
//AB/ End_Object    // prj

//AB-StoreStart
Procedure Prompt_Callback Integer hSL
    Set Initial_Column of hSL to 0
End_Procedure
//AB-StoreEnd
```
After:
```
Object    // prj
End_Object    // prj

Procedure Prompt_Callback Integer hSL
    Set Initial_Column of hSL to 0
End_Procedure
```

RemoveProjectObjectStructure

Removes the legacy *Project Object Structure* comment block at the top of source files, along with the matching `Register_Object` lines. Only `Register_Object` lines for objects listed in the Project Object Structure are removed; other `Register_Object` lines are left untouched. WebApp (`.wo`) files are skipped.

Before:
```
// Project Object Structure
//   BATCH is a dbView
//     Ac_DD is a DataDictionary
//     Art_DD is a DataDictionary
//     ... etc.

Register_Object BATCH
Register_Object Ac_DD
```

```
Register_Object Art_DD
...
```
After:
(all lines removed)

RemoveSansSerif

Removes hard-coded `MS Sans Serif` font lines from source code. Recent DataFlex versions handle fonts considerably better than older releases; explicit `MS Sans Serif` settings usually make modern applications look wrong, so removing these lines lets the current font defaults take effect.

RemoveStudioGeneratedComments

Removes Studio-generated boilerplate comments that no longer carry useful information.
Examples of comments removed:

```
// fires when the button is clicked
//OnChange is called on every changed character
// Visual DataFlex xx.x Client Size Adjuster
// Visual DataFlex xx.x Migration Utility
```

Editor — One File

Functions of type `eEditorFunction` operate on a whole file via the Scintilla editor view. The **Editor** tab page is paged automatically when an Editor function runs, because the Scintilla editor is an OCX component that must be visible to operate.
These functions are slower than line-by-line functions because they round-trip the source through the editor control.

EditorNormalizeCase

Adjusts the case of all scoped words, keywords, and operators to match the casing defined in the editor's language configuration. The mapping is configurable in Editor Settings under **Keywords and Operators** — adjust the spelling of each keyword to match your house style.

Before:

```
procedure OnClick
    integer iRec
    number nOheadOno
    date dDate
    get file_field_current_value of Caltrans_DD)) File_Field Caltrans.Recnum to iRec
    if (nOheadOno = 0) begin
        if (iRec <> 0) Begin
```

After:

```
Procedure OnClick
    Integer iRec
    Number nOheadOno
    Date dDate
    Get File_Field_Current_Value of Caltrans_DD)) File_Field Caltrans.Recnum to iRec
    If (nOheadOno = 0) Begin
        If (iRec <> 0) Begin
```

EditorReIndent

Reindents the whole source file using the configured tab size. Useful for normalizing files that mix indentation styles or that have drifted over time.
Parameter (`sParameter`): indent size in spaces. Valid values: `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`. Default: `4`.
The setting can also be changed in Editor Settings — the two controls are linked.

Before (mixed indentation):

```
    Use POTEXT.DD
    Use ARTSUB.DD

      DEFERRED_VIEW Activate_Art2Auto FOR ;
      Object Art2Auto is a DbView

        Procedure Activating
        Forward send Activating
               Set Value of oform2 to Comp.Mps_PoodAddr
 Set Value of oform3 to Comp.Mps_PopHeadtyp
        End_Procedure
```

After (tab size = 4):

```
Use POTEXT.DD
Use ARTSUB.DD

DEFERRED_VIEW Activate_Art2Auto FOR ;
        Object Art2Auto is a DbView

            Procedure Activating
                Forward send Activating
                Set Value of oform2 to Comp.Mps_PoodAddr
                Set Value of oform3 to Comp.Mps_PopHeadtyp
            End_Procedure
```

Report — One File

Functions of type `eReportFunction` would scan a single file and report findings without modifying source code.
No refactoring functions of this type are currently shipped.

Report — All Files

Functions of type `eReportFunctionAll` scan all selected files together and report findings; they make no changes to the source code. Each function writes its results to a dedicated log file accessible via the **Other Log files** dialog in the toolbar.

ReportUnusedSourceFiles

Reports source files in the workspace that appear to be unreferenced — files that are never the target of a `Use` statement and are not workspace entry points.

After a run, the report is added to the **Other Log files** dialog, and a "Unused Source Files" grid dialog is opened. From the dialog, individual files can be moved to the backup area; the function itself makes no destructive changes.

ReportUnusedUseStatements

Reports `Use` statements that appear to be unnecessary — those that include a package whose classes are never instantiated in the file that uses it.

The function scans each selected source file. For each `Use` statement, it inspects the target package and collects every `Class` definition found there. A `Use` statement is flagged when none of those classes are instantiated by an `Object` declaration in the file. Package files that contain only procedures or functions (no `Class` definitions) are excluded from the check, since such packages are typically used for their side effects.

Results are accumulated across all selected files into a single log file accessible via the **Other Log files** dialog.

Note: This function is conservative. It only reports `Use` statements where the imported classes are unambiguously unused. It does not detect indirect dependencies (a `Use` whose package transitively brings in other necessary code). Review the report manually before removing any `Use` line.

Other — One File

Functions of type `eOtherFunction` operate on a whole file passed as a string array of source lines. The function returns the number of lines affected.

RemoveMultipleBlankLines

Collapses runs of consecutive blank lines down to at most *N* blank lines.
Parameter (`sParameter`): maximum consecutive blank lines. Valid values: `1`, `2`, `3`, `4`, `5`, `6`. Default: `2`.
Note: the engine does not distinguish between object boundaries and method bodies; the maximum applies uniformly throughout the file.

Before:

```
//     order by name;

    Use Windows.pkg
    Use MSSQLDRV.pkg
    Use SQL.pkg



    Set Border_Style to Border_Thick
    Set Label to "Chapter 3 Sample (Microsoft SQL)"
    Set Location to 2 2
    Set Size to 172 299









    Object oResults is a cTextEdit
```

After (max blank lines = 1):

```
//     order by name;
    Use Windows.pkg
    Use MSSQLDRV.pkg
    Use SQL.pkg

    Set Border_Style to Border_Thick
    Set Label to "Chapter 3 Sample (Microsoft SQL)"
    Set Location to 2 2
    Set Size to 172 299

    Object oResults is a cTextEdit
```

RemoveUnusedLocals

Removes variable declarations from functions and procedures when the variable is never referenced inside the function/procedure body.
Known limitations:

- Does **not** remove unused struct-typed variables.
- Two-dimensional arrays are detected for all basic DataFlex types.
- Declarations split across multiple lines (line-continuation) are handled.

- Overlapping variable-name prefixes (e.g. `iSpace` and `iSpaces`) are correctly distinguished.

Returns the number of variable declarations removed.

Other — All Files

Functions of type `eOtherFunctionAll` receive a string array of file paths covering all selected files in the workspace. They are used for cross-file transformations that need a global view.

RestyleDDOs

Renames DataDictionary object declarations across the workspace to a chosen style.

Parameter (`sParameter`): target style. Valid values:

Value	Effect
`eDDStudioStyle`	`o<TableName>DD` (current Studio idiom; default)
`eDDNewStyle`	`o<TableName>_DD`
`eDDOldStyle`	`<TableName>_DD` (pre-Studio legacy)

Default: `eDDNewStyle`.

Background. Before DataFlex 12, DataDictionary objects were declared with bare table names (`<TableName>_DD`). Modern Studio prefixes them with `o`. Mixed styles in the same codebase cause name clashes that the compiler does not catch — copying code from a legacy view into a new dialog can break silently. `RestyleDDOs` normalizes the entire codebase to a single chosen style.

RefactorDbGridTocDbCJGrid

Refactors legacy `DbGrid` (data-aware grid) objects to a CodeJock-based `cDbCJGrid` class with individual column child objects.
Parameter (`sParameter`): a comma-separated list of two or three class names — `OldGridClass,NewGridClass[,NewColumnClass]`. Default: `DbGrid,cDbCJGrid,cDbCJGridColumn`.

Example custom value: `cLangdbGrid,cDbCJGrid,cDbCJGridColumn`. If `sParameter` is empty, any class whose name ends in `dbgrid` is detected and the standard DataFlex classes are used.

For each matching grid object:

- Renames the class to `NewGridClass`.
- Converts the `Begin_Row` / `Entry_Item` list into individual column child objects of type `NewColumnClass`.
- Maps `Set Form_Width Item N to W` → `Set piWidth to W` on the matching column.
- Maps `Set Header_Label Item N to "L"` → `Set psCaption to "L"` on the matching column.
- Removes legacy-only grid properties: `Main_File`, `Wrap_State`, `peResizeColumn`, `psColumnsID`.
- Removes the standard `Entry_Display` boilerplate procedure.
- Preserves all other content (other `Set` properties, nested objects) unchanged.

The scope is code within the legacy object, not the entire file.
Returns the total number of grid objects transformed across all files.

RefactorDbGridTocDbCJGridPromptList

Refactors legacy dbList selection lists to CodeJock-based cDbCJGridPromptList prompt lists.
Parameter (sParameter): a comma-separated list of two or three class names — OldListClass,NewGridClass[,NewColumnClass].

Default: dbList,cDbCJGridPromptList,cDbCJGridColumn. Example custom value: cMyDbList,cDbCJGridPromptList,cDbCJGridColumn. If sParameter is empty, any class whose

name ends in dblist is detected and the standard DataFlex classes cDbCJGridPromptList and cDbCJGridColumn are used.

For each matching dbList object:

- TODO note where the mapping is approximate):
- Auto_Server_State → pbAutoServer
- Auto_Column_State → pbAutoColumn
- Popup_Search_State / Find_Search_State → pbAutoSearch
- Export_Item_State → peUpdateMode (umPromptValue / umPromptRelational)
- Auto_Export_State / Auto_Index_State / Move_Value_Out_State / Auto_Shadow_State / Auto_Label_State → TODO comment only
- Emits a one-time TODO header reminding the developer that the invoking object needs a Prompt_Callback handler (the peUpdateMode pattern).
- The surrounding dbModalPanel and the invoking object are not modified; only the list object itself is transformed.
- Preserves all other content (other Set properties, nested objects, etc.) unchanged.

The scope is code within the legacy object, not the entire file.
Returns the total number of list objects transformed across all files.

RefactorGridToCJGrid

Refactors legacy non-data-aware `Grid` objects (those *not* descended from `DbGrid`) to a CodeJock-based `cCJGrid` class.
Parameter (`sParameter`): a comma-separated list of two or three class names — `OldGridClass,NewGridClass[,NewGridColumnClass]`. Default: `Grid,cCJGrid, cCJGridColumn`. Example custom value: `cMyGrid,cCJGrid,cCJGridColumn`. If `sParameter` is empty, any class whose name ends in `grid` (but **not** `dbgrid`) is detected and the standard DataFlex classes are used.
For each matching grid object:

- Renames the class to `NewGridClass`.
- Translates known legacy property names to their `cCJGrid` equivalents:

Legacy	Replacement
`Set peResizeColumn`	`Set pbAllowColumnResize`
`Set Prompt_Button_Mode`	`Set pbHeaderPrompts`
`Set Allow_Top_Add_State`	`Set pbAllowAppendRow`
`Set Allow_Bottom_Add_State`	`Set pbAllowAppendRow`
`Set Allow_Insert_Add_State`	`Set pbAllowInsertRow`
`Set Auto_Save_State`	`Set pbAutoSave`
`Set No_Create_State`	`Set pbAllowAppendRow` + `Set pbAllowInsertRow` (inverted)
`Set No_Delete_State`	`Set pbAllowDeleteRow` (inverted)
`Set Read_Only_State`	`Set pbReadOnly`
`Set CurrentRowColor`	`Set piHighlightBackColor`

Legacy	Replacement
`Set CurrentCellColor`	`Set piFocusCellBackColor`
`Set CurrentRowTextColor`	`Set piHighlightForeColor`
`Set CurrentCellTextColor`	`Set piFocusCellForeColor`

- Removes legacy-only grid properties: `Wrap_State`, `peResizeColumn`, `psColumnsID`.
- Comments out the body of `Entry_Display`; any color-related `Set` properties inside the original `Entry_Display` are translated and placed above the commented block.
- Comments out other legacy grid methods with a `// TODO:` marker for manual review.
- **Column sub-objects are not generated.** Non-data-aware column structure varies significantly per use case and must be completed manually after refactoring.

The scope is code within the legacy object, not the entire file.
Returns the total number of grid objects transformed across all files.

How It Al lWorks

How It All Works

This page describes the internal architecture of DFRefactor for developers extending the function library. End users do not need to read it.

The Refactoring Engine

The refactoring engine (`cRefactorEngine`, accessed via the global handle `ghoEngine`) is responsible for processing source code. When a session is started, the engine receives a struct containing:

- The selected refactoring functions
- The set of source files to process
- Run-time settings (read-only mode, count-only mode, backup configuration)

`cRefactorEngine` extends `BusinessProcess`. Errors raised during a run are captured and written to the `StatLog` table rather than interrupting the run; they can be reviewed via the **Refactor Engine Error Log** toolbar button.

Core Object Hierarchy

```
cRefactorApplication
    Application startup, settings, and workspace management.

cRefactorEngine                    [ global handle: ghoEngine ]
    Orchestrates a refactoring run: iterates the selected source
    files and calls each selected refactoring function.
      |
      |  calls functions on
      v
oRefactorFuncLib  (object)         [ global handle: ghoFuncLib ]
    The instantiated function library. Contains every public
    refactoring function. New functions are added here.
      |
      |  is a
      v
   cRefactorFuncLib
      Adds {Description} / RegisterInterface meta-tag parsing.
      |
      |  extends
      v
   cBaseFuncLib
      Low-level, private ("_"-prefixed) primitives: the
       tokenizer, expression extraction, and shared helpers.
```

Refactor Function List

The engine scans the oRefactorFuncLib.pkg meta-data on startup. Every published function { Published = True } is collected and automatically updates the **Functions** data table. It then dispatches each call into the `cRefactorFuncLib` repository. That repository extends `cBaseFuncLib`, which provides the low-level tokenizer and a series of system helper functions used by individual refactoring functions.

The most important helper in `cBaseFuncLib` is the tokenizer: given one source line, it produces a struct array of tokens (keyword, identifier, operator, string literal, comment, etc.). Refactoring functions operate against this tokenized view, which is more robust than ad-hoc string matching.

Refactoring function types

Each refactoring function declares its type via a `{ MethodType = ... }` meta-tag in its `{ Description }` block. The engine uses this to decide what shape of data to pass:

MethodType	Input	Per-call scope	Documented on
`eStandardFunction`	`String ByRef sLine, String sParameter returning Boolean`	One source line	Standard — Line by Line
`eRemoveFunction`	`String ByRef sLine returning Boolean`	One source line (for deletions)	Remove — Line by Line
`eEditorFunction`	`String[] ByRef asCode, String sParameter returning Boolean`	Whole file (via Scintilla editor)	Editor — One File
`eOtherFunction`	`String[] ByRef asCode, String sParameter returning Integer`	Whole file (as string array)	Other — One File
`eOtherFunctionAll`	`String[] ByRef asFiles, String sParameter returning Integer`	All selected files at once	Other — All Files
`eReportFunction`	Whole file as string array; *no modifications permitted*	Whole file	Report — One File
`eReportFunctionAll`	All file paths as string array; *no modifications permitted*	All selected files at once	Report — All Files

A `Boolean` return value from a Standard/Remove/Editor function signals whether the line/file was changed. An `Integer` return value from an Other or Report function indicates a count (lines affected, files reported, etc.).

Function parameters

Every refactoring function accepts up to two parameters:

- The **first parameter** always carries the source code (a single line, or an array of lines, depending on `MethodType`).
- The **second parameter (optional)** is `sParameter`, populated from the **Parameter** column of the function list grid. A function can declare valid values for this parameter via the `{ EnumList }`, `{ InitialValue }`, and `{ HelpTopic }` meta-tags.

Adding new functions

The procedure for adding a new refactoring function is:

1. Add the function to `AppSrc/oRefactorFuncLib.pkg` (or `AppSrc/UserDefinedRefactorFunctions.pkg` for personal additions that should not be

exported via the JSON Export/Import dialog).
2. Decorate it with the required meta-tags:
```
{ Published = True }
{ Description = """
    Human-readable description of what the function does.
    { MethodType = eStandardFunction }
    { SummaryText = Short post-run summary text }
    """ }
```
3. Optionally add `{ EnumList }`, `{ InitialValue }`, and `{ HelpTopic }` if the function takes an `sParameter` enum argument.
4. Register the new function in the **Functions** data table via the **TestBench** program.
5. Add unit tests in `oUnit_Tests.pkg` (or a new fixture file in `AppSrc/Tests/`). Run `DFUnit_TestRunner.exe` to verify.

User-defined personal functions in `UserDefinedRefactorFunctions.pkg` can be marked `{ Private = True }` to exclude them from JSON Export/Import.

Note: Need a refactoring function that isn't in the library? RDC Tools International can design, implement, and unit-test one or more for your codebase — an AI-assisted, author-reviewed process. See `SUPPORT.md` in the home folder, or the Feedback page.

Log files

After a refactoring session completes, the engine collects run data into the main log file (`DFRefactoringLogfile.txt`), saved in the workspace's `DFRefactor Backup` sub-folder.
Some functions produce additional log files of their own — for example, `ReportUnusedSourceFiles` writes a list of unreferenced files. These are accessible via the **Other Log files** dialog in the toolbar.
Errors raised during the run are written to the `StatLog` table and accessible via the **Refactor Engine Error Log** toolbar button.

License

DFRefactor is released as freeware under the MIT License. It may be freely used and distributed, but the source code may not be sold or distributed as part of a commercial product.

MIT License

Feedback

Reporting issues

DFRefactor is developed in the open. Bug reports, feature requests, and questions should be filed through the project's GitHub repository:

- **Issues** (bug reports and feature requests): https://github.com/NilsSve/DfRefactor/issues
- **Discussions** (questions, ideas, design conversations): https://github.com/NilsSve/DfRefactor/discussions

Discussion also takes place on the Data Access Worldwide Code Library newsgroup.
For matters that are not appropriate for public forums, email support@rdctools.com.

Commercial services

RDC Tools International offers paid development services for DataFlex codebases, performed by the author of DFRefactor — custom refactoring functions written and unit-tested for your codebase's specific idioms, full legacy-code migrations (performed under strict confidentiality, no source shared with third parties), codebase audits, and team training.

Engagement types, current rates, and how to request a written estimate are maintained in one place so they cannot drift — the project's SUPPORT.md:
https://github.com/NilsSve/DfRefactor/blob/master/SUPPORT.md

Contact

- Bug reports and support questions — file an issue at https://github.com/NilsSve/DfRefactor/issues, or email `support@rdctools.com`
- Commercial inquiries (custom functions, migrations, audits, training) — email `nils.svedmyr@gmail.com`
- Web: http://www.rdctools.com

Disclaimer

Credits

2026 overhauled version:
Nils Svedmyr, RDC Tools International, Sweden.

Testing team:
2024 version:
Sture Andersen (Sture Aps, Denmark),
Yannick Lucassen (Data Access Europe, Netherlands),
Marco Kuipers (28 IT, Australia),
Anderson Rodrigues (2WA, Brazil),
Samuel Pizarro (Brazil),
Raveen Sundram (Excellent Software Ltd, New Zealand),
Michael Mullan (Danes Bridge Enterprises, USA).

Original version: Nils Svedmyr, with contributions from:
Wil Van Antwerpen (Antwise, Netherlands — Scintilla editor integration),
Sean Bamforth,
Allan Kim Eriksen,
Chris Spenser,
Bob Worsley
and Marcia Booth.